

CHAPTER FOUR

System Analysis

4.1 Introduction

This chapter turns the findings of the preceding chapters into a formal specification of the system to be built. Chapter Two established the operational weaknesses of the charitable food sector in Bisha, and Chapter Three confirmed and quantified those weaknesses through a survey of twenty stakeholders. The purpose of this chapter is to analyze the current manual process in detail, describe the system proposed to replace it, define the users the system must serve, and state the functional, non-functional, and technical requirements that follow directly from the evidence already gathered. The chapter then models these requirements formally through a series of UML diagrams, an entity-relationship diagram, and a data dictionary, which together provide the blueprint carried into the design and implementation chapters that follow.

4.2 Current System Description

The process by which surplus food is presently redistributed in Bisha has no dedicated software component. It is carried out entirely through personal coordination among donors, association staff, and volunteers, using telephone calls and informal messaging groups as described in the case study in Chapter Two. A donor becomes aware of a surplus, typically at the close of an event or a business day, and must independently decide whom to contact. If a contact is reached, an association attempts to mobilize a volunteer, who then has to locate the donor's premises without the benefit of a shared map reference and transport the food to a distribution point. Every step in this sequence depends on the availability and responsiveness of a specific individual, so the process has no fallback when a call goes unanswered or a volunteer is already occupied.

The survey results reported in Section 3.4 quantify how often this sequence breaks down. Forty percent of respondents indicated that surplus is simply discarded rather than offered to anyone, and a further thirty-five percent said it is passed on through personal contacts with no documentation of the transaction. Among the respondents who identified a specific weakness

in the current approach, equal proportions pointed to the unavailability of a volunteer at the right time and to the absence of any record of what was donated, while the majority of respondents estimated that thirty minutes or more typically elapse between a donation being reported and a volunteer arriving to collect it. Figure 4.1 traces this workflow step by step and marks the two points, donor-to-association contact and association-to-volunteer dispatch, at which the survey data show delay is most likely to occur.

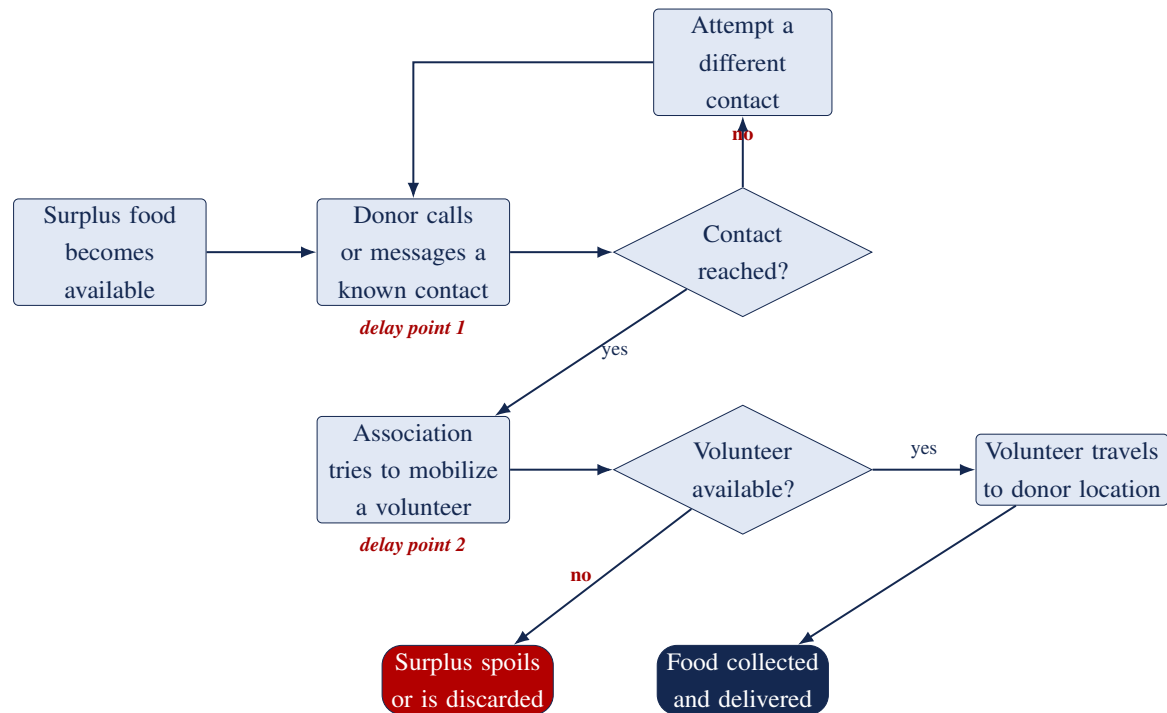


Figure 4.1. Workflow of the current manual food-donation process, with delay points identified from survey data

These characteristics of the current workflow, namely the absence of any persistent record and the lack of a mechanism for matching supply to the nearest available capacity, are addressed directly in the system proposed below.

4.3 Proposed System Description

The proposed system, Al-Khair, replaces the ad hoc sequence described above with a mobile application that gives every participant a defined role and a shared, real-time record of each donation. A donor reports a surplus once, through a short in-app form, and the application immediately determines the donor's location and broadcasts a notification to volunteers within a configurable radius, rather than depending on the donor's personal address book. A volunteer accepts the request from within the application, is guided to the donor by

an embedded map, and confirms collection and delivery, so that the transaction leaves a permanent record from the moment it begins. A charity administrator oversees this activity through a dedicated dashboard that shows active and completed donations, verifies volunteer credentials, and can generate summary statistics for internal reporting. Table 4.1 sets these changes against the current process across the dimensions most affected by the weaknesses identified in Sections 4.2 and 3.4.

Table 4.1. Comparison between the current manual process and the proposed Al-Khair system

Dimension	Current process	Proposed system (Al-Khair)
Response speed	Depends on reaching a known contact by phone; survey data show a typical delay of 30 to 90 minutes or more	Automated, location-based notification to nearby volunteers within seconds of a report
Documentation	No consistent record; most donations are undocumented	Every donation is logged automatically from report to delivery confirmation
Security	No authentication; anyone claiming to be a volunteer may respond	Token-based authentication and role-based access for donors, volunteers, and administrators
Volunteer coordination	Managed informally by association staff, one call at a time	Centralized dispatch with real-time volunteer availability and routing
Donation tracking	Status unknown after a call is placed	Status visible in real time to the donor, the volunteer, and the charity administrator
Reporting for charities	Compiled manually if at all	Automated statistics available through the administrative dashboard

4.3.1 System Users

The system is designed around three primary user roles, each corresponding to a distinct participant in the workflow described above.

Donor. A donor is any household, restaurant, hotel, or wedding hall that reports surplus edible food. Donors register a simple profile, submit donation reports specifying food type, quantity, and readiness time, and receive status updates once a volunteer accepts and completes the collection.

Volunteer. A volunteer is a field participant who collects reported surplus and delivers it to a distribution point on behalf of a partner charity. Volunteers view nearby donation requests, accept or decline them, follow in-app navigation to the donor's location, and confirm collection and delivery within the application.

Charity Administrator. A charity administrator represents a partner organization such as those described in the Chapter Two case study. Administrators approve and manage volunteer accounts, monitor active and historical donations, and access summary statistics used for internal reporting and planning.

A fourth, system-level role, the platform administrator, is responsible for overall account and content moderation and is treated as part of the technical requirements in Section 4.3.4 rather than as a stakeholder-facing role.

4.3.2 System Functional Requirements

The functional requirements below are grouped by module and numbered for traceability to the design and testing chapters that follow. Each requirement is traced to either the case study in Chapter Two or the survey findings in Chapter Three.

Donor Module

FR1. The system shall allow a donor to register and authenticate using a mobile-verified account.

FR2. The system shall allow a donor to submit a donation report specifying food category, estimated quantity, and a readiness time window.

FR3. The system shall automatically capture the donor's location at the time of reporting using device-level GPS.

FR4. The system shall allow a donor to include a simple food-safety confirmation when submitting a report, in direct response to the concern raised in the open-ended survey item reported in Section 3.4.3.

FR5. The system shall notify the donor when a volunteer accepts, arrives, and completes collection of a reported donation.

Volunteer Module

FR6. The system shall send real-time push notifications to volunteers located within a configurable radius of a new donation report, addressing the delay identified at delay point 1 in Figure 4.1.

FR7. The system shall allow a volunteer to accept or decline a donation request from within

the application.

FR8. The system shall provide turn-by-turn map navigation from the volunteer's current location to the donor's location.

FR9. The system shall allow a volunteer to confirm collection and, separately, confirm final delivery to a distribution point.

Charity Administration Module

FR10. The system shall provide a web-accessible dashboard through which a charity administrator can review, approve, or revoke volunteer accounts.

FR11. The system shall allow an administrator to monitor the status of all active and completed donations in real time.

FR12. The system shall generate summary statistics of donation volume over a selectable time period.

FR13. The system shall maintain a permanent, timestamped record of every donation from the moment it is reported, addressing the documentation gap identified in Section 4.2.

4.3.3 Non-Functional Requirements

The non-functional requirements are expressed using the product quality characteristics defined in the ISO/IEC 25010 software quality model (ISO/IEC, 2011), which provides a recognized vocabulary for specifying qualities such as performance efficiency, security, and usability independently of the functional behavior described above.

NFR1 (Security). All authentication shall use signed tokens, and access to donor, volunteer, and administrative functions shall be enforced through role-based access control. Data in transit shall be encrypted using HTTPS.

NFR2 (Performance Efficiency). Notifications of a new donation report shall reach eligible volunteers within a target of five seconds under normal network conditions, consistent with the rapid response the survey identified as the leading priority in Section 3.4.3.

NFR3 (Reliability and Availability). The backend infrastructure shall be hosted on a managed cloud platform to minimize downtime, with a target availability suited to a service used at irregular hours, including late evening periods identified in Section 4.2 as a common time for surplus to arise.

NFR4 (Usability). The interface shall be fully localized in Arabic, follow a minimal-step design for reporting a donation, and be operable by users with varying levels of technical familiarity, in line with the ease-of-use priority identified by respondents in Section 3.4.3.

NFR5 (Maintainability). The application shall be structured into clearly separated modules corresponding to the donor, volunteer, and administrative functions, so that each can be updated independently during future development.

4.3.4 Technical Requirements

The application is developed using the Flutter framework, which allows a single codebase to be deployed to both Android and iOS, consistent with the cross-platform scope defined in Chapter One. The backend is built on Firebase, using Cloud Firestore as the real-time database, Firebase Authentication for identity management, and Firebase Cloud Messaging for push notifications. Location-based routing is provided through the Google Maps Platform, which supplies both the map interface and the turn-by-turn navigation required by FR8.

Figure 4.2 summarizes the resulting data flow at a high level. A user's device communicates directly with the Firebase backend for authentication, data storage, and notification delivery, while location and routing requests are handled through a separate call to the Google Maps Platform. The backend is the single source of truth for donation records, so that the donor, the volunteer, and the charity administrator each observe the same real-time state rather than relying on separately maintained copies of the data.

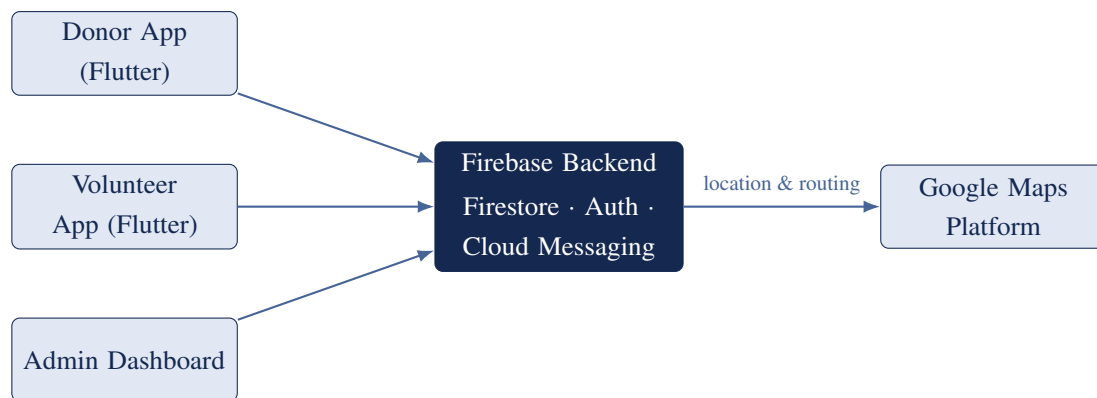


Figure 4.2. High-level data flow between users, the cloud backend, and external services

This architecture keeps the mobile applications free of direct dependencies on one another; the donor and volunteer applications never communicate peer to peer, and every interaction is mediated by the backend, which is what allows the administrative dashboard in Section 4.3.1 to observe a complete and consistent record of activity across the platform.

4.3.5 System Modeling Overview

The requirements defined above are now expressed as a series of formal models using the Unified Modeling Language. Modeling proceeds from the outside in: the context diagram

in Section 4.3.6 fixes the boundary between the system and its external environment, the use case diagram in Section 4.3.7 identifies the discrete interactions each actor has with the system, the activity diagram in Section 4.3.8 traces the end-to-end donation process across all three actors, and the sequence diagrams in Section 4.3.9 detail the message-level behavior of the three processes most central to the requirements in Section 4.3.2, namely authentication, volunteer approval, and the donation transaction itself. Each diagram is cross-referenced to the functional requirement it formalizes.

4.3.6 Context Diagram

Figure 4.3 presents the context diagram of Al-Khair, showing the system as a single process bounded against its four external entities: the Donor, the Volunteer, the Charity Administrator, and the Google Maps Platform identified in Section 4.3.4. Each arrow represents a data flow crossing the system boundary, labeled with the information it carries. Firebase is not shown as an external entity because it constitutes the internal backend of the system rather than an outside actor, consistent with the architecture in Figure 4.2.

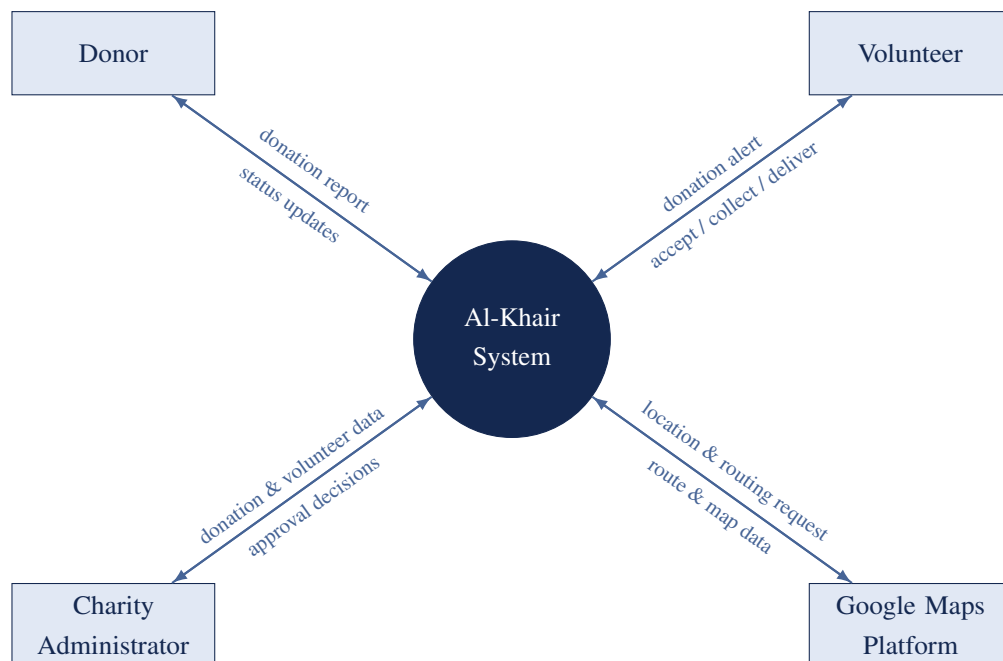


Figure 4.3. Context diagram of the Al-Khair system

4.3.7 Use Case Diagram

Figure 4.4 details the interactions available to each actor. Two structural relationships are used deliberately rather than decoratively. The report submission use case includes food-safety confirmation, reflecting that FR4 is a mandatory part of every report rather than an

optional add-on, and the accept-request use case includes navigation, since a volunteer cannot meaningfully accept a request without being routed to it. By contrast, cancelling a report is modeled as an extension of report submission, since it applies only in the exceptional case where a donor withdraws a report already made.

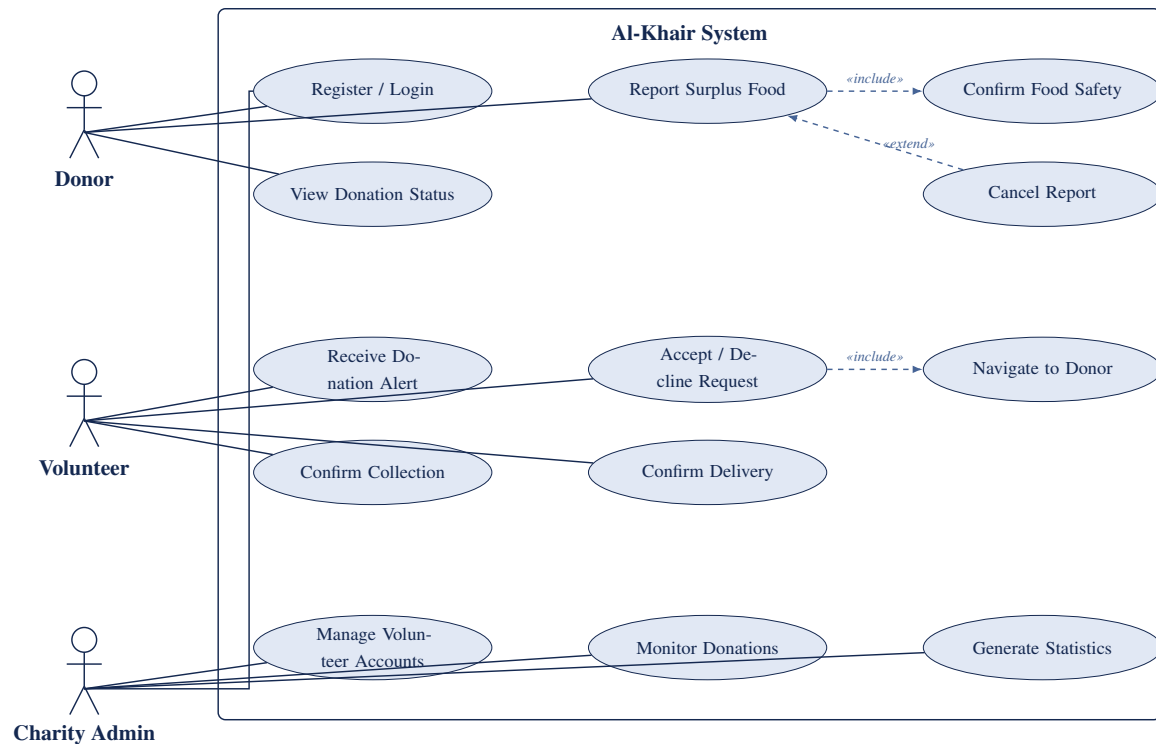


Figure 4.4. Use case diagram for the Al-Khair system

4.3.8 Activity Diagram

Figure 4.5 traces the donation process across all three actors as a single activity diagram organized into swimlanes, from the moment a donor reports a surplus to the moment a charity administrator's records are updated with a completed delivery. The diagram makes explicit the decision points that were left implicit in the manual workflow of Figure 4.1, in particular what the system does when no volunteer accepts a request within the target window defined by NFR2.

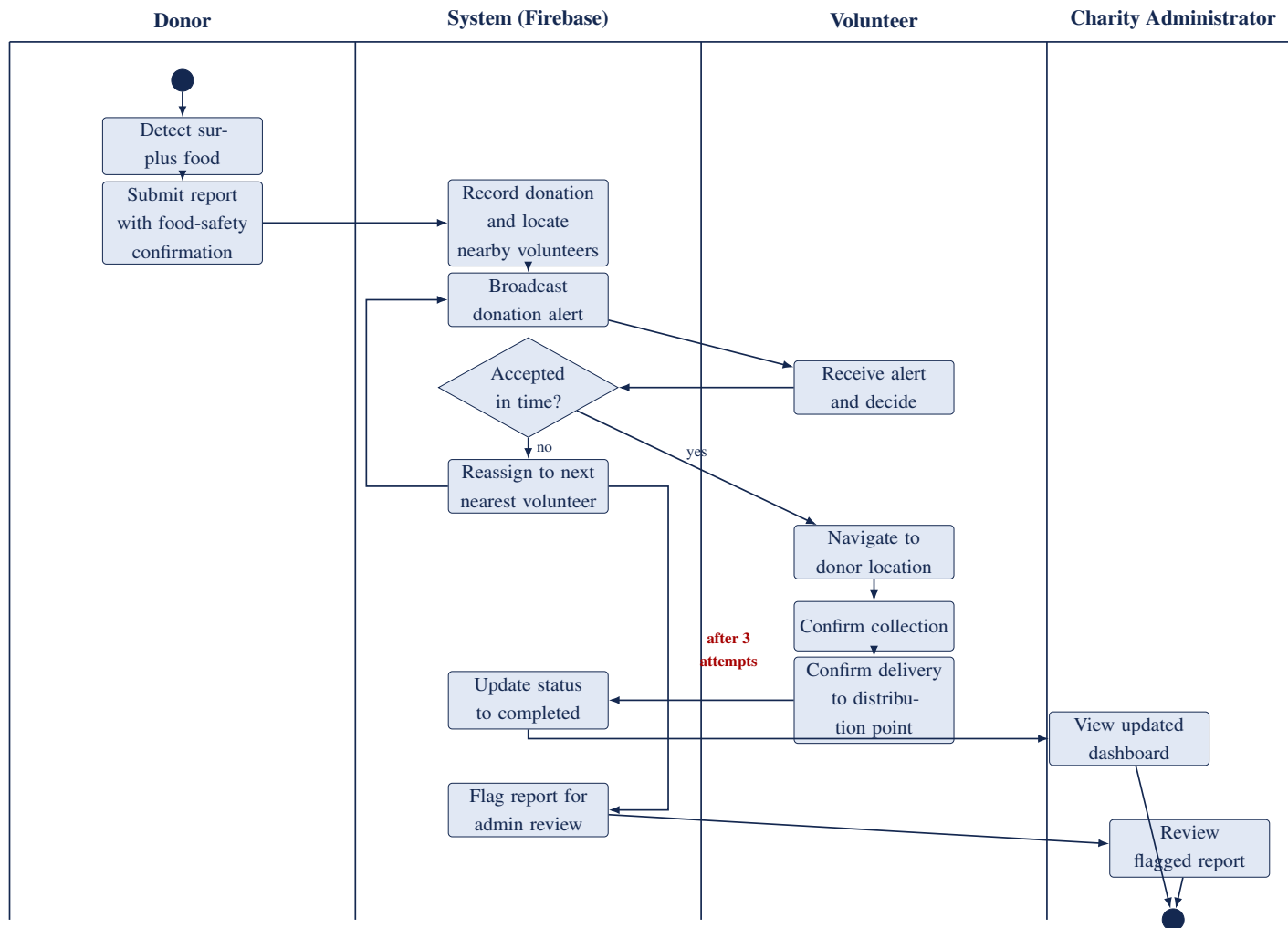


Figure 4.5. Activity diagram of the donation process across donor, system, volunteer, and charity administrator swimlanes

4.3.9 Sequence Diagrams

Three sequence diagrams are presented, covering authentication, volunteer approval, and the donation transaction, in place of a single diagram limited to the donation process alone. Each diagram uses a lifeline per participant and shows only the messages relevant to the corresponding functional requirement.

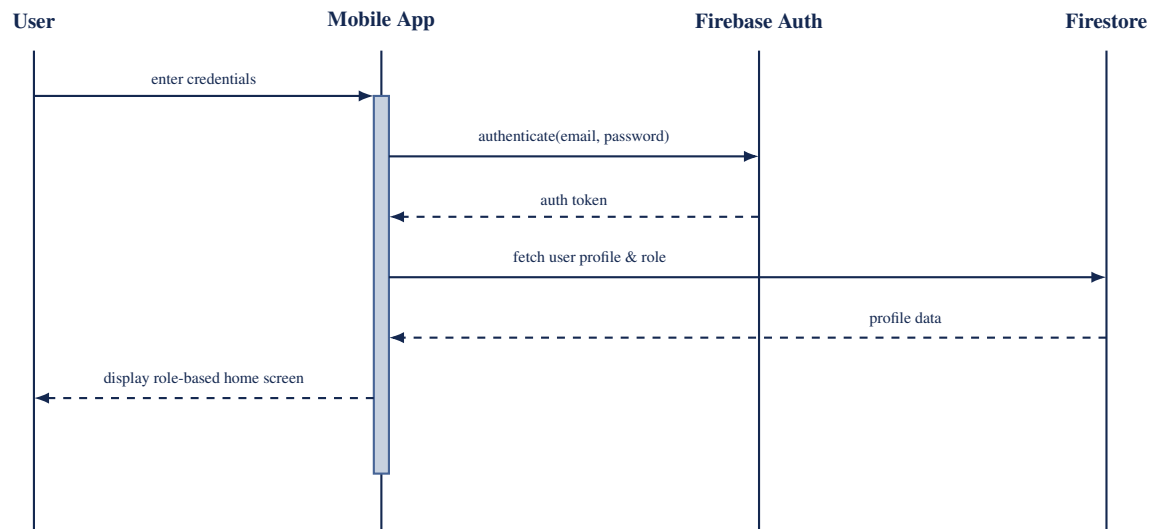


Figure 4.6. Sequence diagram for user authentication (FR1)

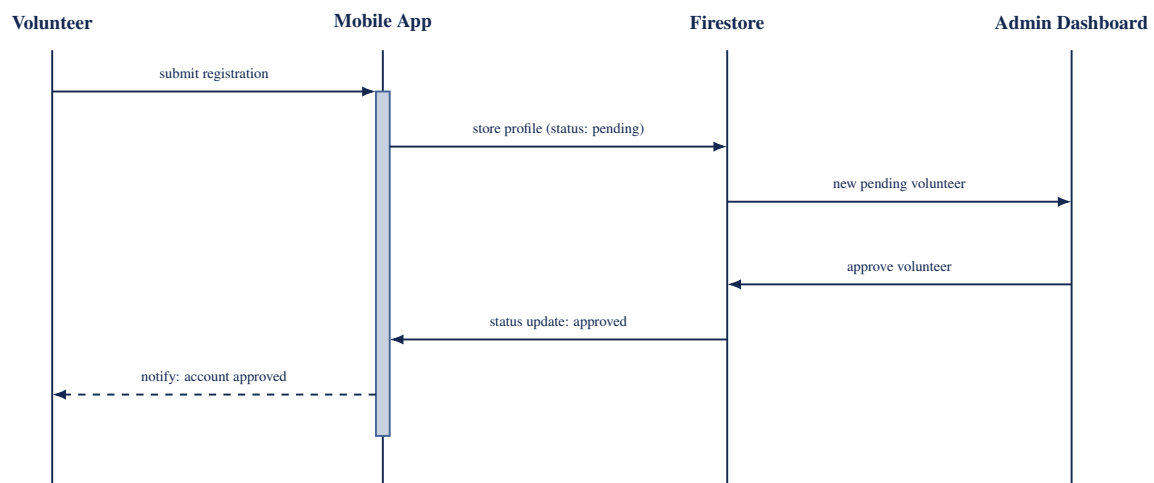


Figure 4.7. Sequence diagram for volunteer account approval (FR10)

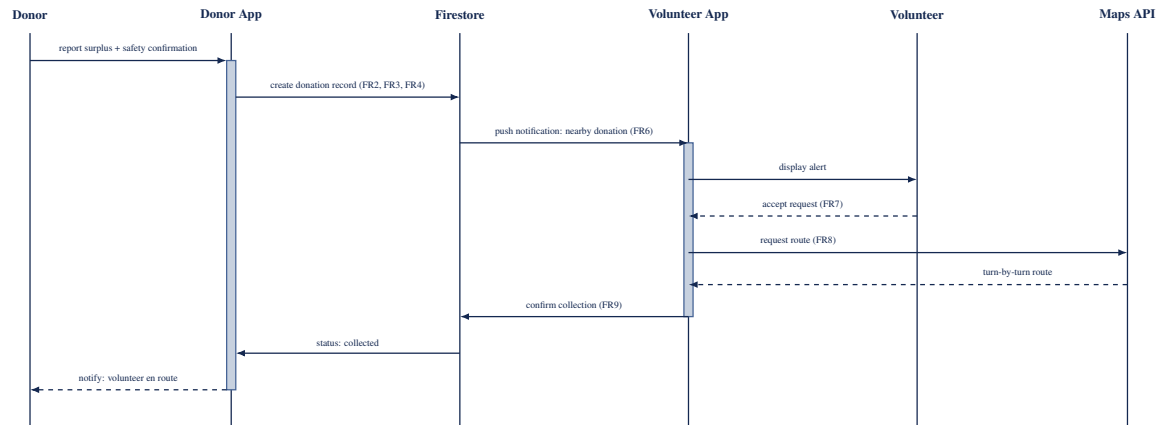


Figure 4.8. Sequence diagram for the donation transaction, from report to collection (FR2–FR9)

4.3.10 Class Diagram

Figure 4.9 models the static structure of the system. Three relationship types are used to reflect genuinely different ownership semantics rather than a single generic association repeated throughout. Generalization captures that Donor, Volunteer, and Charity Administrator are specializations of a common User class that carries authentication and profile behavior. Composition is used between Charity and CharityAdministrator, and separately between Donor and DonationReport, because in both cases the part has no independent existence: a charity administrator record is meaningless without its charity, and a donation report is meaningless without the donor who created it. Aggregation is used between Charity and Volunteer, since a volunteer can exist as a registered user, and in principle move between charities, independently of any single charity's lifecycle. Plain association is used between DonationReport and Volunteer, since a report is assigned to a volunteer without either depending on the other's existence, and between User and Notification for the same reason.

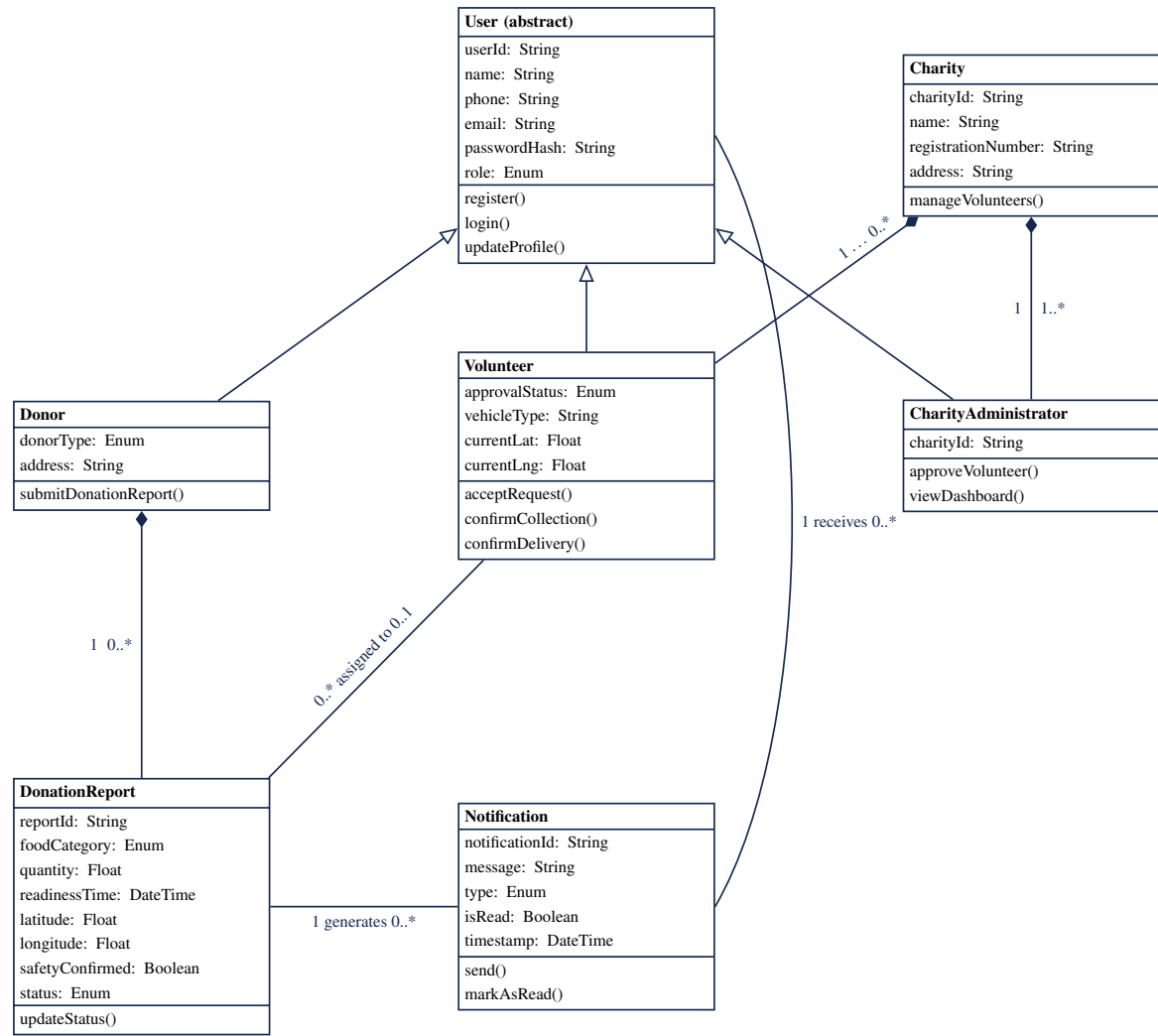


Figure 4.9. Class diagram of the Al-Khair system

4.3.11 Entity-Relationship Diagram

Figure 4.10 translates the class diagram into a relational schema suitable for implementation on Cloud Firestore's document model, expressed here in entity-relationship form for clarity. The inheritance relationships in Figure 4.9 are implemented as separate tables linked back to a single Users table by a shared primary key, a standard relational strategy for representing subtype hierarchies, since Firestore collections are structured to mirror this same separation in practice.

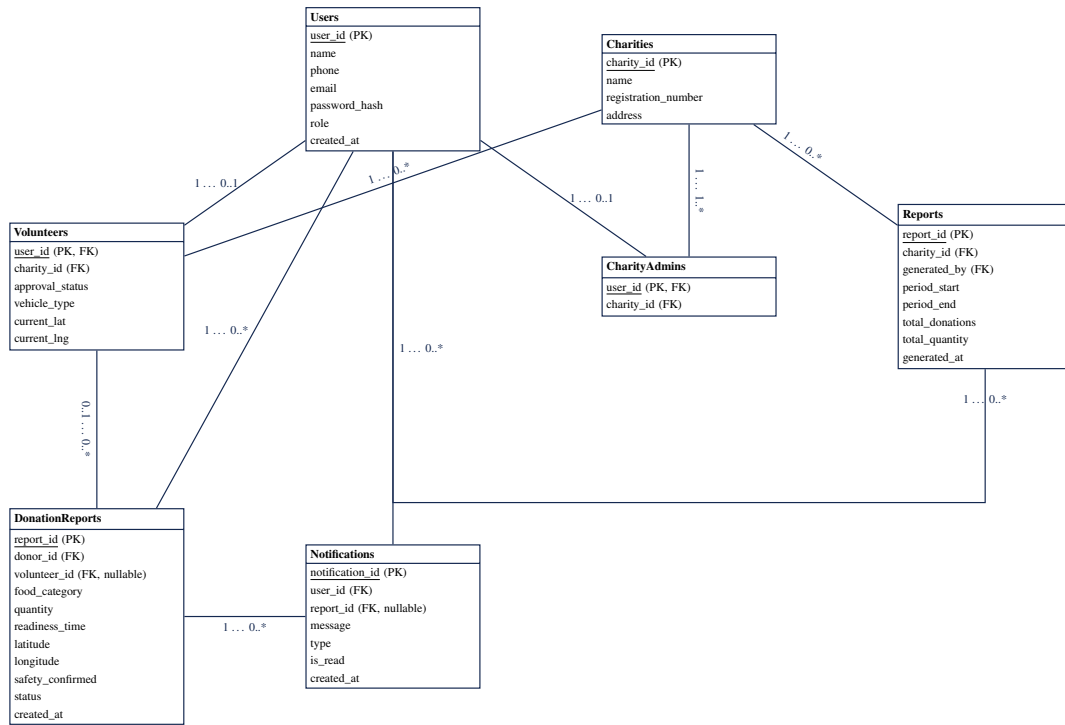


Figure 4.10. Entity-relationship diagram of the Al-Khair database

The schema comprises seven collections in total, a count chosen deliberately to correspond to the seven core, non-report screens presented in the interface design in Chapter Five. The Reports collection is included specifically to back the report-related screens presented there, so that every primary screen in the application maps onto a dedicated collection in the database.

4.3.12 Data Dictionary

Table 4.2 through Table 4.8 document every field in each entity shown in Figure 4.10, giving its data type and a description of its purpose. Primary keys are marked PK and foreign keys are marked FK, consistent with the notation used in the diagram.

Table 4.2. Data dictionary: Users

Field	Type	Description
user_id (PK)	String	Unique identifier assigned to every registered user
name	String	Full name of the user
phone	String	Mobile number, used for verification at registration
email	String	Email address, optional for donors, required for volunteers and administrators
password_hash	String	Securely hashed credential used for authentication
role	Enum	One of donor, volunteer, or charity_admin, used to route the user to the correct interface
created_at	DateTime	Timestamp of account creation

Table 4.3. Data dictionary: Volunteers

Field	Type	Description
user_id (PK, FK)	String	References the corresponding record in Users
charity_id (FK)	String	References the charity the volunteer is affiliated with
approval_status	Enum	Pending, approved, or revoked, set by a charity administrator
vehicle_type	String	Mode of transport used for collection and delivery
current_lat	Float	Volunteer's last reported latitude, used for proximity-based dispatch
current_lng	Float	Volunteer's last reported longitude

Table 4.4. Data dictionary: CharityAdmins

Field	Type	Description
user_id (PK, FK)	String	References the corresponding record in Users
charity_id (FK)	String	References the charity the administrator manages

Table 4.5. Data dictionary: Charities

Field	Type	Description
charity_id (PK)	String	Unique identifier for a partner charitable association
name	String	Official name of the charity, such as the associations listed in Section 2.3.2
registration_number	String	Official registration number with the National Center for Nonprofit Sector
address	String	Physical address of the charity's main distribution point

Table 4.6. Data dictionary: DonationReports

Field	Type	Description
report_id (PK)	String	Unique identifier for a single donation report
donor_id (FK)	String	References the donor who submitted the report
volunteer_id (FK, nullable)	String	References the volunteer assigned to the report, if any
food_category	Enum	Type of surplus food being offered
quantity	Float	Estimated quantity of food, in kilograms or serving units
readiness_time	DateTime	Time at which the surplus is ready for collection
latitude	Float	Latitude of the donation location, captured automatically per FR3
longitude	Float	Longitude of the donation location
safety_confirmed	Boolean	Donor's food-safety confirmation, required per FR4
status	Enum	Reported, assigned, collected, delivered, or expired
created_at	DateTime	Timestamp at which the report was submitted

Table 4.7. Data dictionary: Notifications

Field	Type	Description
notification_id (PK)	String	Unique identifier for a single notification
user_id (FK)	String	References the recipient of the notification
report_id (FK, nullable)	String	References the donation report the notification concerns, if applicable
message	String	Notification text displayed to the user
type	Enum	Category of notification, such as new donation alert or status update
is_read	Boolean	Whether the recipient has viewed the notification
created_at	DateTime	Timestamp at which the notification was generated

Table 4.8. Data dictionary: Reports

Field	Type	Description
report_id (PK)	String	Unique identifier for a single generated report
charity_id (FK)	String	References the charity the report was generated for
generated_by (FK)	String	References the charity administrator who generated the report
period_start	DateTime	Start of the period the report summarizes
period_end	DateTime	End of the period the report summarizes
total_donations	Integer	Count of donations completed within the period, supporting FR12
total_quantity	Float	Total quantity of food recovered within the period
generated_at	DateTime	Timestamp at which the report was generated

4.3.13 Deployment Diagram

Figure 4.11 shows the physical execution environment of the system: the nodes on which each software artifact runs and the communication paths between them. Donor and volunteer devices run the Flutter mobile artifact and communicate with the Firebase cloud environment over encrypted HTTPS, consistent with NFR1, while also calling the Google Maps Platform directly for map rendering and routing. The charity administrator accesses a browser-based build of the same codebase, since Flutter supports web deployment from a single source,

avoiding a separate implementation for the dashboard described in FR10 through FR12.

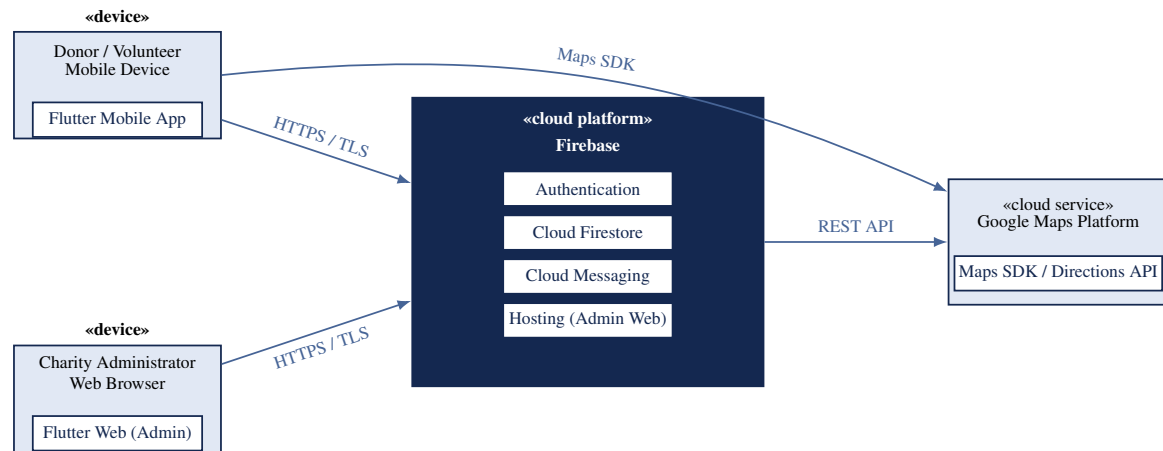


Figure 4.11. Deployment diagram of the Al-Khair system

4.3.14 Component Diagram

Figure 4.12 decomposes the software into components organized in three layers. The presentation layer consists of the three user-facing interfaces identified in Section 4.3.1. Each depends on a corresponding service in the business logic layer through a defined interface, and every service in that layer depends in turn on the Firestore Database component, with the Routing Service additionally depending on the external Maps API component introduced in Figure 4.11.

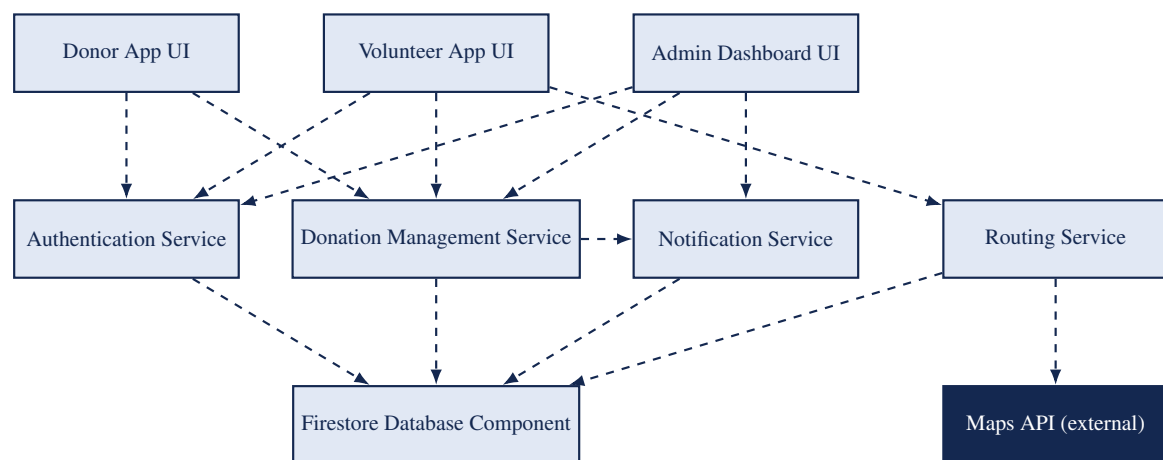


Figure 4.12. Component diagram of the Al-Khair application

4.3.15 Requirement Traceability Matrix

Table 4.9 traces each functional requirement defined in Section 4.3.2 to the diagrams in which it is modeled, confirming that every requirement is represented in at least one behavioral model and one structural model before the design phase begins.

Table 4.9. Requirement traceability matrix

ID	Requirement (summary)	Use Case	Activity	Sequence	Class	Component
FR1	Register and authenticate	✓		✓ (4.6)	✓	✓
FR2	Submit donation report	✓	✓	✓ (4.8)	✓	✓
FR3	Capture donor GPS location		✓	✓ (4.8)	✓	✓
FR4	Food-safety confirmation	✓	✓	✓ (4.8)	✓	✓
FR5	Notify donor of status		✓		✓	✓
FR6	Notify nearby volunteers	✓	✓	✓ (4.8)	✓	✓
FR7	Accept / decline request	✓	✓	✓ (4.8)	✓	✓
FR8	Turn-by-turn navigation	✓	✓	✓ (4.8)	✓	✓
FR9	Confirm collection / delivery	✓	✓	✓ (4.8)	✓	✓
FR10	Approve / revoke volunteers	✓		✓ (4.7)	✓	✓
FR11	Monitor donations in real time	✓	✓		✓	✓
FR12	Generate summary statistics	✓			✓	✓
FR13	Permanent donation record		✓	✓ (4.8)	✓	✓

4.4 Summary

This chapter analyzed the manual process currently used to redistribute surplus food in Bisha, identified the delay points responsible for the outcomes measured in Chapter Three, and specified the Al-Khair system proposed to address them. Three system users were defined, and their needs were expressed as thirteen functional requirements and five non-functional requirements, each traceable to either the Chapter Two case study or the Chapter Three survey findings. These requirements were then modeled formally through a context diagram, a use case diagram, an activity diagram, three sequence diagrams, a class diagram, an entity-relationship diagram spanning seven collections, a data dictionary, a deployment diagram, and a component diagram, with the requirement traceability matrix in Table 4.9 confirming complete coverage across the modeling artifacts. With the system fully specified and modeled, the study proceeds to Chapter Five, which translates this analysis into the architectural and interface design of the application and details its implementation.